

Technical Section

EasySkinning: Target-oriented skinning by mesh contraction and curve editing[☆]Jing Ma^a, Jituo Li^b, Dongliang Zhang^{a,*}^a College of Computer Science and Technology, Zhejiang University, China^b School of Mechanical Engineering, Zhejiang University, China

ARTICLE INFO

Keywords:

Automatic skinning
Mesh contraction
Curve editing
Smooth weight solving

ABSTRACT

Skinning, a critical process in animation that defines how bones influence the vertices of a 3D character model, significantly impacts the visual effect in animation production. Traditional methods are time-intensive and skill-dependent, whereas automatic techniques lack in flexibility and quality. Our research introduces EasySkinning, a user-friendly system applicable to complex meshes. This method comprises three key components: rigid weight initialization through Voronoi contraction, precise weight editing via curve tools, and smooth weight solving for reconstructing target deformations. EasySkinning begins by contracting the input mesh inwards to the skeletal bones, which improves vertex-to-bone mappings, particularly in intricate mesh areas. We also design intuitive curve-editing tools, allowing users to define more precise bone influential regions. The final stage employs advanced deformation algorithms for smooth weight solving, crucial for achieving desired animations. Through extensive experiments, we demonstrate that EasySkinning not only simplifies the creation of high-quality skinning weights but also consistently outperforms existing automatic and interactive skinning methods.

1. Introduction

Skinning, the process that determines how bones affect mesh vertices of the 3D character, plays a crucial role in the animation production process and significantly impacts the final visual effect. However, the traditional weight painting workflow¹ is high in learning cost and time-consuming, often requiring experienced rigging artists to spend weeks to produce high-quality skinned characters. To improve production efficiency, both automatic and interactive skinning technology has seen continuous developing. Nonetheless, existing skinning methods still have limitations in terms of generalizability, ease of use, and deformation quality.

Automatic skinning, an area of ongoing innovation, has evolved through both geometric and data-driven approaches. Geometric methods, pioneered by the work of Pinocchio [1], and later advancements like the Bounded Biharmonic Weights [2] and Geodesic Voxel Binding [3], provide predictable and stable deformations but often lack the flexibility needed for variant shapes. In contrast, data-driven methods, as seen in Neuroskinning [4] and subsequent developments like RigNet [5], NeuralBlendShapes [6], HeterSkinNet [7], and SkinningNet [8], offer adaptability but can suffer from computational intensity and instability in certain scenarios.

Simultaneously, interactive skinning, more commonly adopted in the industry for its high degree of accuracy and flexibility, uses automatic skinning as an initialization step followed by manual refinements. This method provides animators with a precise and controllable interface for modifications through tools like weight painting [9] and curve adjustments [10]. Meanwhile, curve-based editing has emerged as a powerful tool in 3D modeling, offering intuitive control and flexibility. Techniques such as Teddy [11], FiberMesh [12], and EasyToy [13] demonstrate the benefits of using curves for designing and editing 3D models, yet their potential in character skinning has not been fully explored.

Previous skinning methods are summarized in Table 1 using the “initialize (segmentation) then optimize (diffusion)” approach, which reveals that no method satisfies all three criteria of generalizability, ease of use, and accuracy. Our research introduces EasySkinning, a user-friendly skinning approach applicable for multi-component, intersecting, and non-manifold meshes, which effectively addresses the robustness and precision issues prevalent in existing techniques. EasySkinning’s methodology comprises three core components.

Rigid weight initialization by mesh contraction. This process begins with Voronoi-guided mesh contraction, leveraging its ability of

[☆] This article was recommended for publication by Marcel Campen.

* Corresponding author.

E-mail address: dzhang@zju.edu.cn (D. Zhang).¹ <https://www.youtube.com/watch?v=0foI4-ZOInc>

Table 1

Comparisons of skinning methods. (A) Initialization phase. A.1: compute vertex-to-bone distance, A.2: find seed vertices for heat diffusion, A.3: identify influential region. (B) Optimization phase. B.1: compute smooth weight according to distance, B.2: heat diffusion from seed vertices, B.3: manual, B.4: solve smooth weight to reconstruct target deformations. (C) Support multi-component, intersecting, and non-manifold meshes. (D) Interaction intensity: more plus sign + means more user interactions, minus sign - means no user interaction.

Method	A	B	C	D
Closest Distance	A.1 euclidean distance d_{ij} from vertex to bone	B.1 $w_{ij} = f(d_{ij})$	✓	-
Heat Diffusion [1]	A.2 vertices st. $d_{ij} \leq \epsilon$	B.2 harmonic diffusion from seed vertices	✗	-
BBW [2]	A.2 vertices st. $d_{ij} \leq \epsilon$	B.2 biharmonic diffusion from seed vertices	✗	-
GVB [3]	A.1 geodesic voxel distance	B.1 $w_{ij} = f(d_{ij})$	✓	-
Neural Skinning [4,5,7,8]	A.1 euclidean [4]/geodesic [5]/hollow distance [8]	B.1 $w_{ij} = f(d_{ij})$	✓	-
Weight Painting	A.3 initial weight from auto-skinning method	B.3 manual	✓	+++
ARAPLBS [14]	A.3 rigid weight by thresholding smooth weight	B.4 solve smooth weight to reconstruct ARAP deformations	✓	++
Spline Interface [10]	A.2 spline editing to specify seed vertices	B.2 harmonic + biharmonic diffusion from seed vertices	✗	++
Ours	A.3 contracted-based segmentation + curve editing	B.4 solve smooth weight to reconstruct target deformations	✓	+

moving vertices inward to the bones to establish an accurate correspondence between the skeleton bones and mesh vertices. This is particularly effective in densely packed areas such as fingers, armpits and hips, where current automatic algorithms often fall short. **Rigid weight editing with curves.** Following the initial weight assignment, EasySkinning empowers users with intuitive curve-editing tools suitable to complex production meshes. These tools allow for the refinement of rigid weights through various curve operations such as sketching, merging, splitting splines, drawing extension lines, and adding or deleting control points. This step further improves the precision of the influential region for each bone. **Smooth weight solving for reconstructing target deformations.** The final stage involves solving for smooth weights, essential for realistic deformation of the animated character. We propose a general goal-oriented approach that derives smooth weights from target deformations. Initially, based on the refined rigid weights from the curve-editing step, we employ advanced deformation algorithm such as Delta Mush and Direct Delta Mush to acquire high-quality deformed shapes. Subsequently, we use SSDR algorithm to solve smooth weights in order to reconstruct the target shapes by Linear Blend Skinning technique, ensuring high-fidelity animation and real-time deployment.

In the subsequent sections, we delve deeper into EasySkinning's methodology. We highlight the crucial roles of mesh contraction and curve editing in improving initialization accuracy and target-oriented smooth weight solving for achieving the desired animation effect. A thorough evaluation, including user studies and comparative analyses, has validated the effectiveness of our approach, demonstrating its significant advantages in character skinning. In summary, our contributions are three folds:

- We improve the accuracy in the initialization (segmentation) stage through mesh contraction and curve editing.
- We introduce a general framework for the optimization (diffusion) stage, deriving smooth weight to replicate target deformations.
- We implement an easy-to-use skinning system suitable for production meshes containing multi-components, self-intersections, and non-manifold primitives.

2. Related work

2.1. Skinning

The field of automatic skinning has seen significant developments over the years, with progress made in geometry-based and data-driven methods, as shown in the first five rows of Table 1.

Pinocchio proposed by [1], laid the groundwork for automatic skinning of 3D characters using geometric approaches. This approach offers stability and predictability, with its deterministic nature decided by the heat-diffusion equation, refer to Eq. 1 in [1]. However, the method

can struggle with complex or irregular shapes. Advancements in geometric methods have led to more refined techniques. For instance, the Bounded Biharmonic Weights (BBW) method proposed by [2] generates smoother skinning weights by replacing the Laplacian operator with bi-Laplacian operator. Yet another milestone is the Geodesic Voxel Binding (GVB) method proposed by [3,15], which improves the handling of production character meshes that might be irregular. Despite their improvements, geometric methods still face challenges in generalizability and adaptability, areas where data-driven methods have shown promise.

Data-driven methods for automatic skinning adopt a unique approach by learning skinning weights from data. Neuroskinning of [4] introduces the use of neural graph networks for automatic skin binding, offering adaptability in irregular meshes that geometric methods often lack. More recent works like RigNet of [5], NeuralBlendShapes of [6], HeterSkinNet of [7], and SkinningNet of [8] have expanded the capabilities of data-driven methods. They utilize advanced neural networks to predict skinning weights and animate articulated characters. However, these methods have limitations. While they are generalizable, they often result in unstable results that necessitate substantial manual refinement. Additionally, they depend on extensive training data and require significant computational resources.

Beyond automatic skinning methods, semi-automatic (interactive) skinning techniques are more prevalent in the industry due to their higher accuracy. The most commonly adopted approach is weight painting [9], where initial skinning weights computed by automatic methods are refined by artists using brush tools. However, the artist must iteratively try and test to ensure the deformations are plausible, a process that is tedious and requires considerable time and experience. Similarly, ARAPLBS [14] uses automated methods to set initial weights and then updates these weights to reconstruct As-Rigid-As-Possible deformations. The system provides users with pre-processing and post-processing tools to ensure the method works for production meshes and generates desired skinning weights. Another method, Spline Interface [10], offers an intuitive user interface that employs spline curves to set up and modify skinning weights, but it is limited to single-component meshes, whereas most production characters are multi-component.

We have summarized the aforementioned methods in Table 1. It is evident that no method currently satisfies the criteria of generalizability, ease of use, and accuracy simultaneously. Our proposed approach aims to address these challenges by providing a generalized, user-friendly, and precise skinning method that effectively overcomes the limitations of existing techniques.

2.2. Skeleton extraction by mesh contraction

Mesh contraction, a key concept in computer graphics as introduced in the paper of [16], plays a crucial role in providing a compact representation of a 3D object's morphology. It is employed in geometric processing [17], shape analysis [18], and physical simulations [19],

etc. This process involves strategically moving the vertices inward, thereby establishing a contracted mesh with skeleton-like structure. One seminal method [16] of achieving this is through implicit Laplacian smoothing [20]. This approach contracts the mesh geometry towards a skeletal shape, preserving the topology of the original mesh while ensuring the vertices are correctly aligned with the skeleton. However, a challenge in this method is the potential drifting from the medial center. An advanced technique, Mean Curvature Skeletons (MCS) proposed by [21], tackles this by introducing an energy term using mesh's Voronoi poles to guide the formation of a medially centered structure. The process includes local remeshing on the surface mesh through edge splits and collapses, simplifying mesh connectivity and reducing numerical instability from triangulation degeneracies. Despite these improvements, MCS alters the vertex count and can lose original topology information.

Our approach contracts the mesh towards skeletal bones while preserving the original edge connections. This method provides invaluable information on vertex-to-bone correspondence, significantly improving the prediction accuracy of skinning weights in complex areas such as fingers, armpits, and thighs where vertices with different semantics are closely situated.

2.3. Curve-based editing

Curve-based editing has provided significant advantages in designing and manipulating 3D models. It allows for intuitive and flexible control over the shapes, enabling artists to create complex designs with relative ease.

Teddy of [11] presents a user-friendly approach for designing cartoon characters using editable curves. This method allows users to sketch out the toy design in a series of curves, which are used to generate a 3D model. The main advantage of this approach is the intuitive interface and the freedom it gives to the user in designing the 3d models. However, it is primarily suitable for relatively simple shapes and may struggle with more complex designs. FiberMesh of [12] introduces a novel technique that employs 3D curves for designing freeform surfaces. It provides a mechanism for users to create, manipulate, and design complex 3D models using a network of curves, referred to as the FiberMesh. This method offers fine control over the design process, allowing the creation of intricate and complex shapes. In addition to curve-based modeling, CurveNet of [22] introduces a novel approach to character articulation, where detail-preserving deformations are entirely driven by 3D curves profiling the deforming surface. This method initiates with a spline-based rigging system, empowering artists articulate the character with flexible curve controls.

Our curve tools are mainly inspired by above three works and the Spline Interface [10]. Instead of applying curves to create shapes or to drive animation, we replace the tedious process of brushing skin weights with the various curve-editing operations, which is demonstrated to be more robust and effective on production meshes.

3. Overview

In the EasySkinning method, we introduce an innovative approach of skinning characters. Beginning with an input character (Fig. 1a), our pipeline initiates with the contraction of mesh vertices and the computation of initial rigid weights (Fig. 1b). Compared to the original mesh, this contracted mesh more accurately establishes the binding relationship between vertices and bones, particularly effective in densely packed mesh areas where previous methods [3–5,8,15] potentially struggle. Following this, we provide users with intuitive curve-editing tools (Fig. 1c), allowing them to refine rigid weights for more precise bone influence. This includes operations like dragging, inserting, and removing control points of the curve, and the creation of new curves for refining influence regions. Once these modifications are made, users can directly assign weights by clicking on the bounded

Table 2
Frequently used symbols.

Symbol	Description
n	Number of mesh vertices
B	Number of skeleton bones
$\mathbf{v}_i$	Position of the i th vertex under rest-pose
$\check{\mathbf{v}}_i$	Vertex position after mesh contraction
$\tilde{\mathbf{v}}_i$	Vertex position deformed by DeltaMush
$\hat{\mathbf{v}}_i$	Vertex position deformed by LBS
$\mathbf{o}_j$	Position of the j th bone's head under rest pose
w_{ij}	Influence of the j th bone on the i th vertex
$\mathbf{w}_i$	Bone influence weights for the i th vertex
$\mathbf{W}^r$	Rigid weight matrix $w_{ij}^r \in \{0, 1\}$
$\mathbf{W}$	Predicted smooth weight matrix $w_{ij} \in [0, 1]$
$\mathbf{W}^s$	Ground truth smooth weight matrix
$\ \cdot\ _1$	L1 norm
$\ \cdot\ $	L2 norm

region and selecting the relevant bone. The final step involves the computation of smooth weights. We first utilize the Delta Mush (DM) algorithm [23] to generate high-quality deformation sequences, and then employ the SSDR algorithm [24] to solve smooth weights, aiming to reconstruct target deformations. This process ensures that the final skinning weights produce the desired deformation the artist expects. Overall, EasySkinning offers a comprehensive and intuitive solution for character skinning, simplifying the intricate process while ensuring high-quality results. We detail our method in subsequent sections, covering rigid weight initialization (Section 4.1), curve-based editing (Section 4.2), and smooth weight solving (Section 4.3).

Table 2 outlines the symbols frequently used in this paper. Scalars are denoted in italic type, vectors in boldface, and matrices in boldface capital letters. For instance, the weight matrix $\mathbf{W}$ for a character with n mesh vertices and B skeleton bones, is represented by n rows, each corresponding to the weight vector $\mathbf{w}_i$, with a total of $n \times B$ weight values w_{ij} .

4. Method

4.1. Rigid weight initialization by mesh contraction

In skeletal animation, accurate weight assignment is crucial for achieving realistic deformations. Previous methods of weight assignment often struggle with the dilemma of accurately distinguishing between vertices associated with distinct bones, especially when these vertices are spatially close. These challenges can result in deformation artifacts that significantly reduce the quality of the animation, often necessitating extensive post-processing for correction. We find that mesh contraction can address these challenges by enabling a more accurate vertex-to-bone mapping. This technique involves moving mesh vertices inward to the skeletal bones, leading to improved bone segmentation (more accurate vertex-to-bone mapping).

We propose a Voronoi-guided contraction method, adapted from [16,21], for initializing rigid weights. This method begins with an input mesh of n vertices ($\mathbf{V}$) and seeks to find the contracted mesh vertices ($\check{\mathbf{V}}$) that balance contraction, attraction, and medial forces. The contraction energy compels vertices to move inward along the inverse normal direction, ensuring better alignment with the corresponding bones. Meanwhile, the attraction energy anchors vertices to their original positions, acting as a counterbalance to the contraction energy so that the mesh does not collapse during the contraction process. The medial energy encourages vertices to move towards mesh medial centers, locations where skeleton bones usually occur. Our objective is to minimize the total energy E combining contraction, attraction, and medial energies, represented mathematically as

$$E = E_{\text{contraction}} + E_{\text{attraction}} + E_{\text{medial}}$$

$$= \|\mathbf{L}\check{\mathbf{V}}\|^2 + \sum_i (w_{ii}^a)^2 \|\check{\mathbf{v}}_i - \mathbf{v}_i\|^2 + \sum_i (w_{ii}^m)^2 \|\check{\mathbf{v}}_i - \mathbf{u}_i\|^2 \quad (1)$$

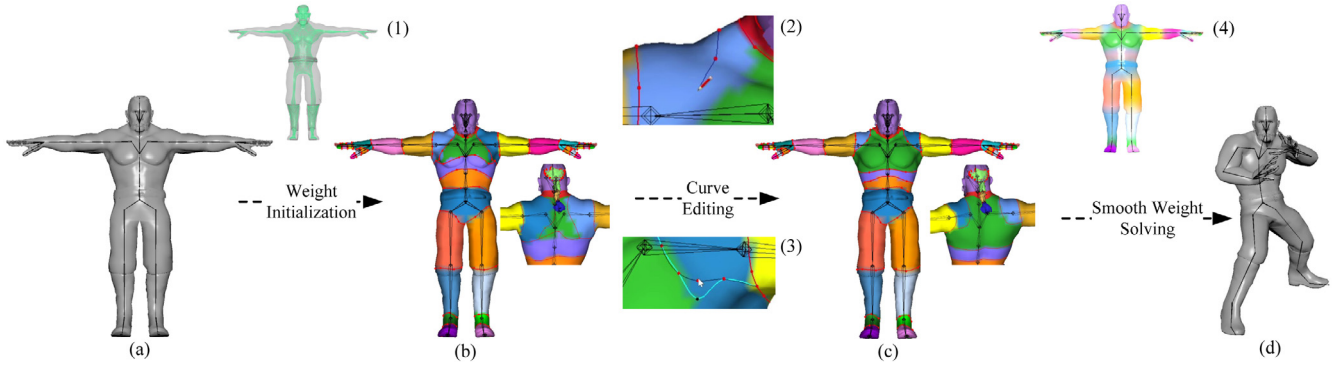


Fig. 1. EasySkinning overview. Starting with an input character (Brute) shown in (a), our pipeline begins by contracting mesh vertices (1) and computing the initial rigid weights (b), namely, identify influential/segmentation region of each bone. Next, we empower users with intuitive curve-editing operations (2, 3), enabling them to obtain refined rigid weights (c). Finally, we complete the process by solving for smooth weights (4), which are essential for achieving desired deformations (d).

Minimizing E equates to solving a linear system

$$\begin{bmatrix} \mathbf{L} \\ \mathbf{W}^a \\ \mathbf{W}^m \end{bmatrix} \tilde{\mathbf{V}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{W}^a \mathbf{V} \\ \mathbf{W}^m \mathbf{U} \end{bmatrix} \quad (2)$$

where $\mathbf{L} \in \mathbb{R}^{n \times n}$ is the discrete Laplace operator [20], a crucial component in the process, as it provides a measure of surface curvature and facilitates the movement of vertices towards the skeleton. $\mathbf{U} \in \mathbb{R}^{n \times 3}$ represents the Voronoi poles associated with the original mesh vertices, indicating the medial axis of the mesh. $\mathbf{W}^a$ and $\mathbf{W}^m$ are diagonal weighting matrices that control the degree of attraction to the previous vertex positions and the speed of convergence towards the medial center respectively, with the diagonal values w_{ii}^a and w_{ii}^m being the weight to i th vertex. The exact values of $\mathbf{W}^a$ and $\mathbf{W}^m$ as well as the validation of Voronoi mesh contraction are detailed in Section 5.1.1. We solve Eq. (2) using the least squares method, as it is an overdetermined system. Specifically, we solve the equation $\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{3n \times n}$ is the matrix on the left-hand side composed of $\mathbf{L}$, $\mathbf{W}^a$, and $\mathbf{W}^m$, and $\mathbf{b} \in \mathbb{R}^{3n \times 3}$ is the right-hand side matrix. The solution matrix $\mathbf{x} \in \mathbb{R}^{n \times 3}$ represents the contracted mesh vertices $\tilde{\mathbf{V}}$ that we aim to solve for. To compute $\mathbf{x}$, we employ direct LDL^T Cholesky factorization on $\mathbf{A}^T \mathbf{A}$, which allows us to efficiently solve for the contracted vertices.

The contracted vertices are then utilized to calculate the initial rigid weights, as shown in Fig. 2b (upper right corner). Given the contracted vertices $\tilde{\mathbf{V}}$ and the skeleton composed by multiple bone segments $\mathbf{o}_{ij}$, where $\mathbf{o}_i$ is i th joint's position, we compute the shortest distance between each vertex $\tilde{\mathbf{v}}_k$ to a specific bone segment using the following equation

$$d = \begin{cases} \|\tilde{\mathbf{v}}_k - \mathbf{o}_i\| & \text{if } \mathbf{o}_{ij} \cdot (\tilde{\mathbf{v}}_k - \mathbf{o}_i) < 0 \\ \|\tilde{\mathbf{v}}_k - \mathbf{o}_j\| & \text{elif } \mathbf{o}_{ji} \cdot (\tilde{\mathbf{v}}_k - \mathbf{o}_j) < 0 \\ \|\tilde{\mathbf{v}}_k - \mathbf{o}_i\| \sin \alpha & \text{else} \end{cases} \quad (3)$$

where α is the angle between bone segment $\mathbf{o}_{ij}$ and the vector $\tilde{\mathbf{v}}_k - \mathbf{o}_i$, refer to Fig. 2b. If the projection of $\tilde{\mathbf{v}}_k$ onto the line extending from $\mathbf{o}_i$ to $\mathbf{o}_j$ falls before $\mathbf{o}_i$, the distance d is the Euclidean distance from $\tilde{\mathbf{v}}_k$ to $\mathbf{o}_i$. If the projection falls beyond $\mathbf{o}_j$, the distance d is the Euclidean distance from $\tilde{\mathbf{v}}_k$ to $\mathbf{o}_j$. Otherwise, the vertex projects onto the bone segment itself, and the distance d is the perpendicular distance from $\tilde{\mathbf{v}}_k$ to the line defined by $\mathbf{o}_{ij}$. After computing these distances, vertices are rigidly bound to their nearest bone, yielding the initial rigid weight matrix $\mathbf{W}'$, with $\mathbf{W}'_{ij} = 1$ indicating that j th bone is closest to i th vertex.

Rather than relying on the original mesh vertices for computing vertex-to-bone distances, employing the contracted vertices results in a significantly more accurate assignment of rigid weights. This approach is particularly effective in complex regions of the mesh where vertices are spatially proximate yet anatomically belonging to different bones, as illustrated in Fig. 2 and Section 5.1.1. In such regions, previous methods might struggle to correctly assign weights, often leading to

unrealistic deformations during animation. However, with the inward vertices, there is a clear delineation of bone influence, which further improves the final deformation quality.

4.2. Rigid weight editing with curves

The initial rigid weights obtained through the contraction-based method may not always capture the rigger's intent precisely. Consequently, riggers need to refine the bone influential regions. In traditional processes, riggers adjust these regions manually through a method known as weight painting. This technique, while effective, is notably time-consuming and labor-intensive. Inspired by the paper [10, 13], we propose a curve-editing approach that simplifies the adjustment of influential regions to be as straightforward as sketching/modifying curves. The use of editable curves introduces a more intuitive and efficient method for defining bone influence.

4.2.1. Curve initialization

Spline generation. Our method automatically generates the initial boundary curves among different influential regions (red curves in Fig. 1b). We use cubic Hermite splines to represent the editing curves due to its flexibility for user interactions. The spline is a type of piecewise polynomial function passing through a given set of data points [25,26], which is particularly advantageous for its versatility and smoothness properties. It is commonly used in numerous fields such as computer graphics [22], image processing [27], and geometric modeling [13]. The mathematical formulation of the cubic Hermite spline involves a series of basis functions, each of which influences only a segment of the curve. Each segment of the cubic Hermite spline curve interpolates two control points and their tangent vectors, mathematically defined as

$$H(t) = (2t^3 - 3t^2 + 1)\mathbf{p}_0 + (t^3 - 2t^2 + t)\mathbf{g}_0 + (-2t^3 + 3t^2)\mathbf{p}_1 + (t^3 - t^2)\mathbf{g}_1 \quad (4)$$

where t is a parameter that varies from 0 to 1 along the curve segment, $\mathbf{p}_0$ and $\mathbf{p}_1$ are the start point and the end control point of the spline segment respectively, and $\mathbf{g}_0$ and $\mathbf{g}_1$ are the tangent vectors at these control points. By manipulating the control points of the splines, riggers can easily adjust the influential region a particular bone exerts over the mesh.

For the construction of initial boundary curves among different influential regions, our approach begins with identifying all edges that lie along the boundary between two regions, forming a contiguous strip of edges. The midpoints of these edges are considered as potential control points for the spline. To determine an optimal subdivision length (the distance between two control points), we calculate the average bone radius r of the involved regions, and the bone radius in each region is the average vertex-to-bone distance. Setting the subdivision length to

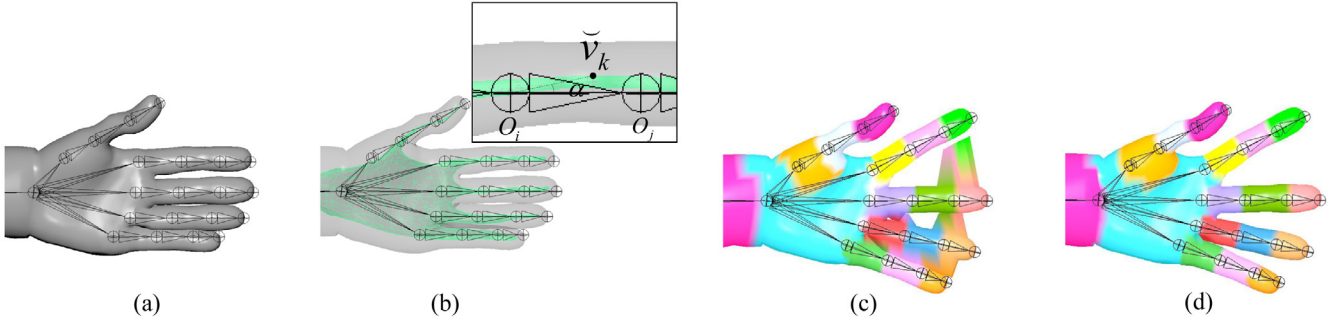


Fig. 2. Weight initialization by mesh contraction. Our method shows remarkable accuracy in complex regions, such as fingers (a). This is achieved by mesh contraction (b), which reposition mesh vertices towards the interior, closer to the bones. Unlike the standard closest-distance method (c), our contraction-based approach determines vertex assignments to bones by evaluating the distances between the contracted vertices and the bones. This approach allows for a clear distinction in areas that are in close proximity yet controlled by different bones (d).

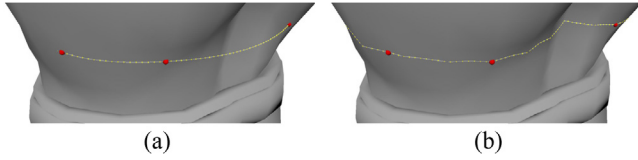


Fig. 3. Curve initialization. (a) Spline generation: interpolating curve points (yellow) according to control points (red) and Eq. (4). (b) Spline projection: projecting interpolated points onto the mesh surface.

the average bone radius has proven generally effective for facilitating user adjustments, considering that the circumference of a circle is $2\pi r$. This typically results in approximately six control points for a closed curve, more details about subdivision length is given in Fig. 8. We then select control points from these midpoints at intervals of subdivision length. Employing the cubic Hermite spline formula of Eq. (4), the curve points between two control points are interpolated, as shown in Fig. 3a.

Spline projection onto surface. After interpolation, we map the interpolated points onto the surface mesh. Given that the control point's corresponding face is already known, our algorithm initially attempts a direct projection onto the last successfully mapped face. This projection occurs along the inverse direction of the average normal vector from the two control points' corresponding faces. If a suitable mapping point is not found on the current face, the algorithm iteratively searches adjacent faces until it locates an appropriate mapping point. Each time a suitable mapping point is found, the corresponding segment of the spline curve is adjusted to ensure that the curve smoothly conforms to the mesh surface. Ultimately, this process results in a set of curve points that adhere closely to the mesh surface, representing the boundaries of the bone influence region, as shown in Fig. 3b.

4.2.2. Rigid weight editing

We introduce a user-friendly curve-editing functionality that enables precise modification of rigid weights. Unlike weight painting, which requires users to adjust weights brush by brush, this feature provides an intuitive way to modify curves by dragging, inserting, or removing control points. It also offers users the versatility to create new curves using various methods such as drawing extension lines, as well as sketching, merging, or splitting splines.

Inserting a new control point into an existing spline is straightforward: the user simply moves the cursor to the desired location and clicks to add the point. This action is visually represented in Fig. 4a. Internally, our algorithm updates the spline by adding the new control point, recalculates the spline to reflect the change, and remaps it onto the mesh surface to maintain accuracy. Similarly, removing or moving a control point involves selecting the desired point for adjustment,

with the system automatically updating and recalculating the spline as needed, as illustrated in Fig. 4b and Fig. 4c.

In addition to modifying curves, our system supports multiple methods for generating new curves. Users can establish straight lines by selecting start and end points, with the system employing a shortest-path algorithm to trace the line across the mesh surface. They can also draw extension lines across the mesh, with the curve automatically created over the mesh based on the intersection with a plane defined by the extension line and the user's view direction, as demonstrated in Fig. 4e. For sketching a spline, users can dynamically create a new curve by adding each control point, as shown in Fig. 4d. Moreover, users can merge multiple splines or split a single spline, adapting the curve to their specific needs. The algorithm for creating splines through merging and splitting is similar to the one described in the last paragraph.

Following these modifications, the resultant influential regions are precisely defined by these curves, as depicted in Fig. 4f. Users can effortlessly re-assign weights by clicking an interior point within the bounded area and selecting the desired bone. This action triggers the automatic reallocation of the rigid weight to the chosen bone, a process further visualized in the supplementary video. Our algorithm initiates this procedure by intersecting the mesh faces intersected by the curve. Starting from the seed point selected by the user, it employs a breadth-first search algorithm to accurately classify each vertex as either inside or outside the region enclosed by these curves. This process continues until all vertices within the bounded region are identified, ensuring more precise rigid weights for the downstream task.

4.3. Smooth weight solving

Following the computation of the rigid weights, our method integrates advanced deformation methods that can be derived from skeletal animations to generate the optimal shapes. We examine three deformation methods in Section 4.3: As Rigid As Possible (ARAP) [28], Delta Mush (DM) [23], and Direct DeltaMush (DDM) [29]. We incorporate both DM and DDM into the proposed system due to their generalizability (apply to production meshes) and high-quality results. Artists can choose between DM and DDM for their projects, with DM set as the default option. Our algorithm automatically rotates each bone around the x, y, and z axes between $-\pi/2$ and $\pi/2$ with a step size of $\pi/3$ to generate the skeletal animations. By combining DM/DDM, we acquire the optimized deformations.

With these optimized shapes as input, we introduce the SSDR algorithm to solve smooth skinning weights. SSDR is an automated algorithm to extract linear blend skinning (LBS) parameters (smooth skinning weight and per-frame bone transformations) from a sequence of deformed shapes [24]. LBS is currently the most widely adopted

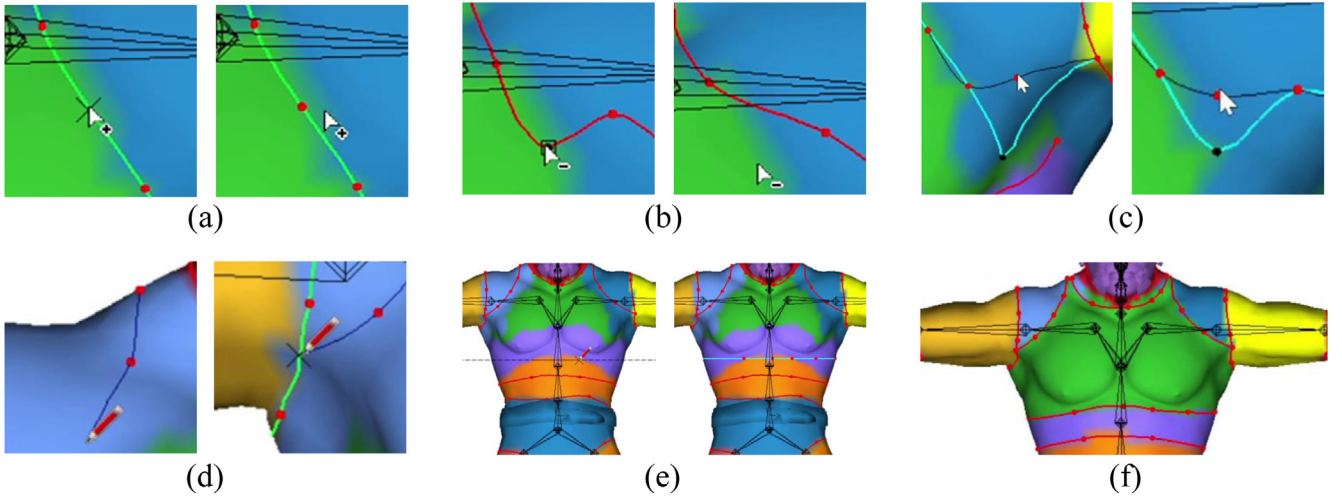


Fig. 4. Curve editing. We demonstrate the process of refining influential regions (Fig. 1b) into a more precise version (Fig. 1c). The user begins by adjusting the spline curve on the left shoulder by inserting (a), removing (b), repositioning (c) control points as indicated in the first row. Then, she sketches a new spline curve on the right shoulder (d) and draws an extension line intersecting the character's upper body (e). Finally, the user assigns mesh vertices to the appropriate bone by clicking a point inside boundary curves and selecting the corresponding bone, as illustrated in (f). More details are presented in the supplementary video.

technique in real-time applications such as games and virtual reality. The mathematical formulation of the LBS model is as follows:

$$\hat{\mathbf{v}}_i = \sum_j w_{ij} (\mathbf{R}_j \mathbf{v}_i + \mathbf{t}_j) \quad (5)$$

where $\mathbf{v}_i$ represents i th vertex's rest-pose position, and $\hat{\mathbf{v}}_i$ denotes the deformed position of the i th vertex, influenced by the transformation (rotation $\mathbf{R}_j \in \mathbb{R}^{3 \times 3}$ and translation $\mathbf{t}_j \in \mathbb{R}^3$) of the j th bone, with w_{ij} being the weight of the j th bone's influence on the i th vertex.

Given a sequence of mesh vertices animated by an advanced deformation algorithm driven by skeletal animations, the SSDR algorithm seeks the smooth weight matrix $\mathbf{W}$ that minimizes the discrepancy between the target vertex positions and the vertex positions calculated by LBS. The optimization objective is captured as

$$\min_{\mathbf{W}} \sum_t \sum_i \|\tilde{\mathbf{v}}_i^t - \hat{\mathbf{v}}_i^t\|^2 \quad (6)$$

where $\tilde{\mathbf{v}}_i^t$ is the vertex position generated by DM/DDM for i th vertex at time t , and $\hat{\mathbf{v}}_i^t$ is i th vertex's position derived from LBS at time t . By substituting $\hat{\mathbf{v}}_i^t$ with the expression from Eq. (5), the optimization equation expands to

$$\min_{\mathbf{W}} \sum_t \sum_i \left\| \tilde{\mathbf{v}}_i^t - \sum_j w_{ij} (\mathbf{R}_j^t \mathbf{v}_i + \mathbf{t}_j^t) \right\|^2 \quad (7a)$$

Subject to:

$$w_{ij} \geq 0, \forall i, j \quad (7b)$$

$$\sum_{j=1}^B w_{ij} = 1, \forall i \quad (7c)$$

$$|\{w_{ij} | w_{ij} \neq 0\}| \leq K, \forall i \quad (7d)$$

where $\mathbf{v}_i$ is the rest-pose position of i th vertex, $\mathbf{R}_j^t$ and $\mathbf{t}_j^t$ denote the rotation and translation applied to j th bone at time t , respectively. The total number of bones is represented by B . And K serves as a limit on the number of bones that can influence a single vertex, ensuring the solution's sparsity, which is usually set to four in real-time application. The optimization framework guarantees that the weights are non-negative (Eq. (7b)), sum to one for each vertex (Eq. (7c)), and adhere to a sparsity constraint (Eq. (7d)). To solve Eq. (7), we use the Active Set Method (ASM) [30] to handle non-negativity and affinity constraints, ensuring the weights are non-negative and sum to one.

Initially, we address the sparseness constraint by associating a limited subset of bones with each vertex. The solver iteratively updates the weights by identifying active constraints and adjusting the solution to minimize the least squares error. Finally, we select the most significant weights, ensuring the solution is sparse and optimal.

By introducing a general framework that solves smooth to approximate target deformations, our pipeline achieves a high-fidelity representation of deformed shapes, which are driven by LBS and solved smooth weights, making them suitable for real-time applications.

5. Experiments

5.1. Implementation details

Our method is developed in C++ and Maya. It has been rigorously tested on a Windows 10 laptop powered with an Intel i9-13900HX 2.2 GHz Processor and 16 GB of RAM. The performance of our program is optimized for interactivity, as demonstrated in the accompanying video.

Dataset. For our experiments, we collect a range of rigged 3D characters from Mixamo² and Free3D.³ These characters, known for their high quality and versatility, provide a robust foundation for our tests, as shown in Fig. 5 and Table 3. As part of our experimental setup, we standardize each character model by scaling it to a uniform bounding box with a maximum diagonal length of 200 units, and we adjust their vertical position to align the lowest point to be at the floor. This normalization allows for consistent comparisons and analysis. Furthermore, to evaluate the effectiveness of our skinning approach, we animate the characters using a collection of motion data obtained from Mixamo and from animations created by skilled animators. This enables us to thoroughly test the quality and functionality of our skinning process under various movement scenarios.

Having outlined the experimental setup and data sources, we will now proceed to discuss the key components of our method in greater detail.

² <https://www.mixamo.com/>

³ <http://thefree3dmodels.com>

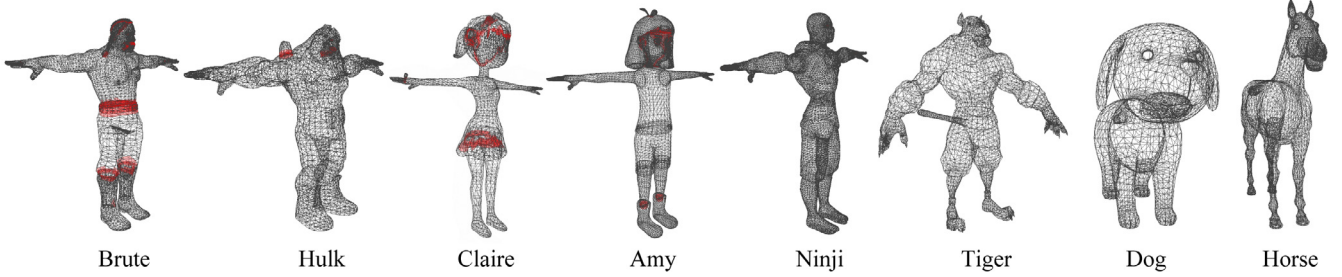


Fig. 5. Statistics of input characters are shown in Table 3. Intersecting faces are shown in red.

Table 3

Statistics of input characters. #V: #Vertices, #J: #Joints, #F: #Faces, #C: #Components, #I: #Intersecting Faces, #NM: #Non-manifold primitives.

Name	#V	#J	#F	#C	#I	#NM
Brute	14 338	72	24 747	336	3858	0
Hulk	4858	65	9620	18	662	0
Claire	4224	87	8183	20	1130	6
Amy	12 456	65	24 568	11	1498	4
Ninji	12 465	65	24 780	3	506	0
Tiger	3980	71	7956	1	0	0
Dog	2606	36	5176	1	78	0
Horse	5783	47	11 538	1	0	0

5.1.1. Rigid weight initialization

In this section, we examine five approaches for automatically generating initial rigid weights: *Closest Distance* (no mesh contraction), *Argmax GVB* [3,14] (no mesh contraction), *Projective Skinning* [19], *Standard Contraction* [16], and our adapted *Voronoi Contraction* from [21].

- The *Closest Distance* method uses the original mesh vertices to compute the distance to the skeleton bones. Each vertex is then assigned to the closest bone based on the minimum distance.
- The *Argmax GVB* approach is similar to the initialization method proposed in ARAPLBS (see Sec 3.1 of [14]), which computes smooth weights first and then identifies bone's influential region by thresholding/argmax the smooth weights. Specifically, we employ the GVB algorithm [3] to calculate the initial smooth weights, assigning each vertex to the bone with the highest influential weight.
- The *Projective Skinning* method, illustrated in Fig. A.21, moves vertices towards their nearest base points on the non-volumetric skeleton. Laplacian smoothing is then applied to these base points, averaging them with neighboring vertices to resolve misalignments commonly found on complex surfaces. Afterward, vertex-to-bone assignments are determined by casting rays from the mesh vertices to their smoothed base points and identifying intersections with the volumetric skeleton. This method is detailed in Fig. 2 and Section 3.2 of [19].
- The *Standard Contraction* method moves vertices inwards pulled by the contraction and the attraction forces, and then uses the contracted vertices for distance calculation.
- Our adapted *Voronoi Contraction* also uses the contracted vertices for distance computation, but it integrates an additional medial energy, which directs the vertices towards the Voronoi poles. For details on how the Voronoi poles are computed, please refer to Appendix B. The sensitivity analysis of our adapted algorithm is discussed in Appendix C.

Table 4 displays the Rigid Weight Accuracy (RWA) of these methods. The RWA is the number of correctly binded vertices relative to the number of mesh vertices. A vertex is considered correctly binded

Table 4

Rigid Weight Accuracy (RWA) of different methods. (Unit: %).

Contraction type	Avg RWA	Min RWA	Max RWA
Closest Distance	88.62	83.33	94.22
Argmax GVB [3,14]	87.20	67.28	94.71
Projective Skinning [19]	85.02	81.01	89.98
Standard Contraction [16]	89.43	84.33	95.81
Voronoi Contraction	90.79	85.54	96.93

if it is binded to a bone that is within the set of influential bones. Mathematically, the RWA is defined as:

$$RWA = \frac{\sum_i \mathbb{I}((\mathbf{w}_i^r > 0) \cap (\mathbf{w}_i^g > \epsilon))}{n} \quad (8)$$

where $\mathbb{I}((\mathbf{w}_i^r > 0) \cap (\mathbf{w}_i^g > \epsilon))$ is the indicator function that evaluates to 1 if the i th vertex is binded to the correct bone, and 0 otherwise. By summing over all vertices in the mesh, we obtain the number of correctly binded vertices. The variable n represents the number of mesh vertices. The vector $\mathbf{w}_i^r = [w_{i1}^r, w_{i2}^r, \dots, w_{iB}^r]$ denotes the one-hot rigid weight vector of the i th vertex, which is generated from Section 4.1 or Section 4.2. This vector identifies the bone to which the vertex is rigidly binded, with only one element non-zero and equal to 1, representing the binded bone. The vector $\mathbf{w}_i^g$ represents the ground weights for the i th vertex, downloaded from the Mixamo&Free3D website. The parameter ϵ is the weight threshold for a bone to be considered influential to a vertex. We set $\epsilon = 0.25$ since only a maximum of four bones are allowed to influence a vertex in most applications. Consequently, $\mathbf{w}_i^g > \epsilon$ results in a binary vector where ones indicate the influential bones for the i th vertex. Table 4 demonstrates that employing mesh contraction generally leads to better results compared to no contraction, with Voronoi contraction outperforming standard contraction. This observation is further supported by Fig. 6 and Fig. 7. Contraction-based methods attract vertices towards the skeleton, resulting in fewer wrongly binded vertices. However, how the vertex is contracted inward significantly impacts the final results, specifically how to determine the contraction direction and strength. *Projective Skinning* exhibits poor results due to inaccuracies in determining contraction directions. Initially, it uses the *Closest Distance* approach to assign vertices to corresponding bones (deciding the inward direction). Even the subsequent Laplacian smoothing of base points does not adequately rectify this issue, and it even makes the situation worse, leading to notable misbindings, as illustrated in hairs and fingers of Fig. 7a. In the case of *Standard Contraction*, vertices are moved along the inverse direction of their normals using the uniform contraction strength. Compared to *Projective Skinning* and *Standard Contraction*, *Voronoi Contraction* determines the contraction direction and strength based on Voronoi poles, which achieves a more precise vertex-to-bone correspondence (see Fig. 7c).

For the contraction methods of Eq. (2), we initialize the attraction coefficient w_{ii}^a to 0.5 and the medial coefficient w_{ii}^m to 0.1 for all n vertices. Unlike Tagliasacchi et al. [21], our adapted Voronoi Contraction foregoes remeshing operations such as vertex removal or edge collapse.

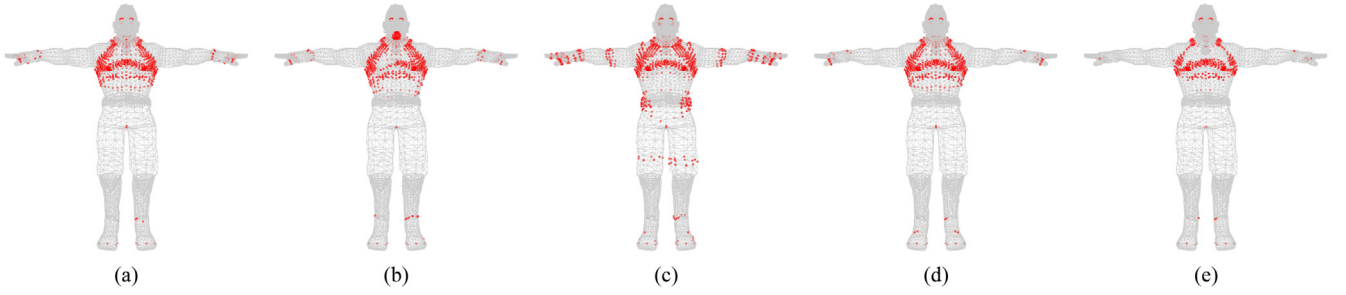


Fig. 6. Comparison of different initialization methods for rigid weight. (a) Closest Distance (no mesh contraction), (b) Argmax GVB [3,14] (no mesh contraction), (c) Projective Skinning [19], (d) Standard Contraction [16], (e) our adapted Voronoi Contraction from [21]. If a vertex is binded to a bone that is in the set of influential bones, it is regarded as correctly binded. Vertices incorrectly binded to the wrong bone are highlighted in red point.

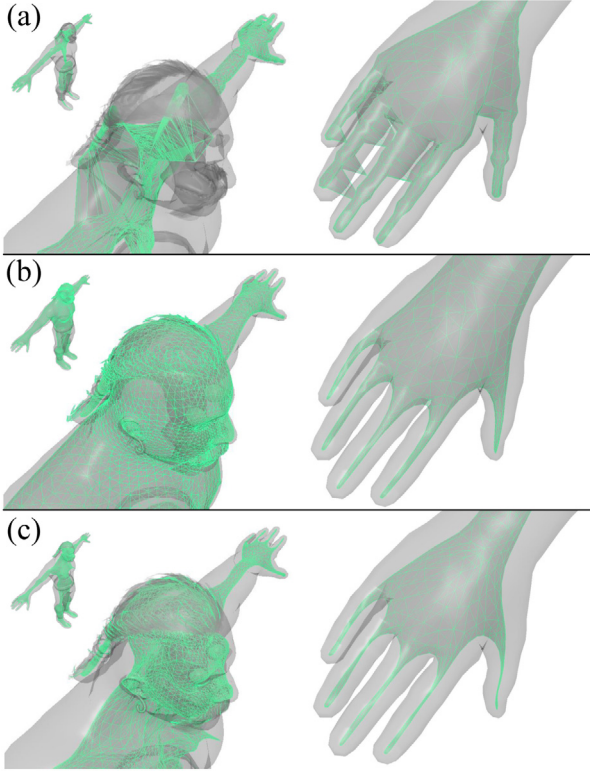


Fig. 7. Comparison of different contraction method. (a) Projective Skinning [19]. (b) Standard Contraction [16]. (c) Our adapted Voronoi Contraction from [21].

This approach ensures the preservation of mesh integrity, thereby ensuring that each vertex can be mapped to the corresponding bone. However, this method underperforms in the case of coarse meshes, as discussed in Appendix C.

5.1.2. Curve editing

In the context of curve-based weight editing, the number of control points on the spline is crucial for balancing ease of use and precision. Excessive control points can overwhelm users, complicating the editing task, while too few may compromise the detail and accuracy of the adjustments. As shown in Fig. 8, the density of spline control points is inversely proportional to the subdivision length; therefore, selecting a subdivision length within the range of $[0.5r, r]$ is generally optimal. Accordingly, our system sets the default subdivision length to r , meanwhile, providing users the flexibility to add or remove control points for tailored control.

To assess the impact of the curve-editing phase on Rigid Weight Accuracy (RWA), we compared the RWA before and after curve adjustments, as illustrated in Table 5. The results, collected from our

Table 5

Rigid Weight Accuracy with or without curve editing (Unit: %).

Curve editing	Avg RWA	Min RWA	Max RWA
✗	90.79	85.54	96.93
✓	96.92	93.16	99.15

user study (detailed in Section 5.2), reveals improvements in Average RWA, Minimum RWA, and Maximum RWA, with the most significant enhancement observed in the Minimum RWA. This improvement demonstrates the effectiveness of our curve-editing functionality in addressing challenging cases that are not optimally resolved by the weight initialization algorithm.

5.1.3. Smooth weight solving

We evaluate three deformation methods: ARAP [28], DM [23], and DDM [29], to explore their effectiveness in mesh deformation and smooth weight solving.

- **ARAP (As-Rigid-As-Possible)** [28] ARAP is renowned for maintaining shape rigidity during mesh deformation, facilitating interactive manipulation while preserving original surface characteristics through ARAP energy minimization. Initially, the transformations of boundary vertices are specified. The positions of the remaining vertices are then determined iteratively by local-global optimization. In our implementation, we start with a rigid weight initialization as detailed in Section 4.1. We then identify boundary vertices by selecting the middle vertices of each bone. This selection is achieved by casting rays from the midpoint of each bone to intersect with the mesh faces. The positions of these vertices are set according to Eq. (5) and rigid weights. Subsequently, the positions of the remaining vertices are solved to minimize ARAP energy while maintaining the specified boundary conditions.
- **DM (Delta Mush)** [23] stands out for its superiority in allowing riggers to use simpler skinning weights such as rigid weights to produce realistic deformations. During the pre-processing time, DM employs Laplacian smoothing to filter out the high-frequency detail and caches the details (delta). During the runtime, it runs the Laplacian smoothing algorithm on the deformed mesh to acquire a smoothed geometry (mush), and then adds the details back on top of the mush to get the final shape. In our experiment, we set the smoothing iterations as 20 and the smoothing strength as 0.5.
- **DDM (Direct Delta Mush)** [29] extends DM by providing an even more efficient and controllable approach to mesh deformation. DDM eliminates the iterative runtime smoothing steps of DM, and allows users to control bending and twisting deformations flexibly. In our experiment, we configure DDM with parameters as follows: 20 smoothing iterations ($p = 20$), a rotation smoothness of 1.0 ($\lambda = 1.0$), a translation smoothness of 0.8 ($\kappa = 0.8$), and a blending weight of 0.9 ($\alpha = 0.9$), as detailed in Section 3.4 of [29].

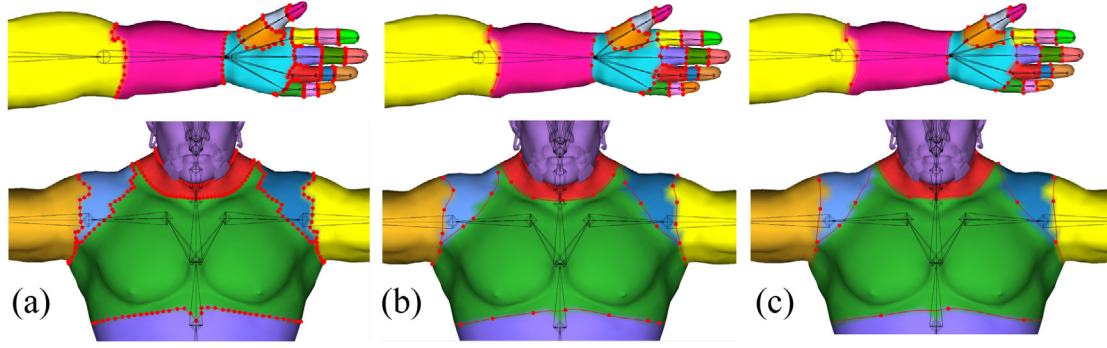


Fig. 8. Illustration of curve control points at varying subdivision lengths. (a) 0, (b) $r/2$, (c) r , where r is the average bone radius of the involved regions. The bone radius for each influential region is calculated as the average vertex-to-bone distance within that region.

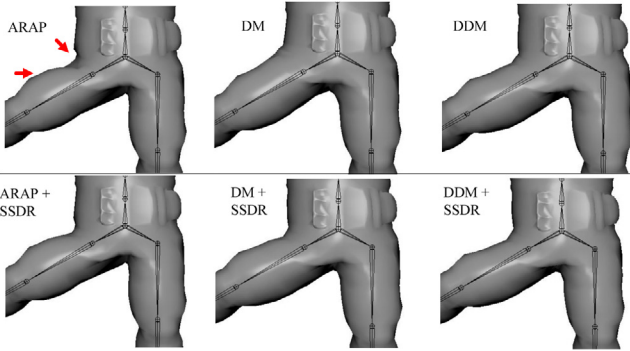


Fig. 9. Hulk's upperleg. Deformation of ARAP [28], DM [23], and DDM [29] (1st row) and LBS results using the smooth weights solved by SSDR (2nd row).

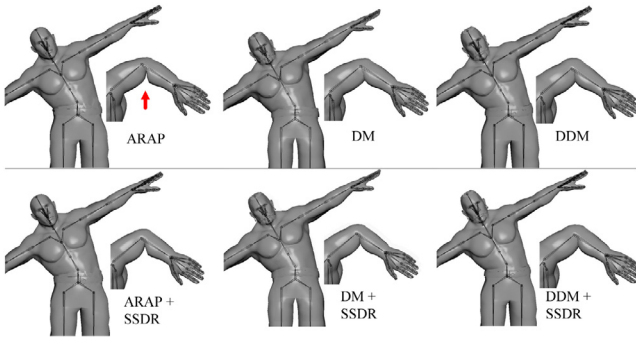


Fig. 10. Brute's spine and elbow. Deformation of ARAP [28], DM [23], and DDM [29] (1st row) and LBS results using the smooth weights solved by SSDR (2nd row).

The deformation results of these three methods and the solved smooth weight by Eq. (7) are shown in Figs. 9 and 10. For the Hulk and Brute models used in ARAP algorithm, we have processed the original meshes to ensure they are watertight. As depicted in the first row of Figs. 9 and 10, both the Delta Mush (DM) and Direct Delta Mush (DDM) methods effectively mitigate the bulging and cleft artifacts prominent in ARAP deformations. The second row demonstrates that the smoothed weights, when applied in conjunction with Linear Blend Skinning (LBS), nearly replicate the original deformations, validating the efficacy of the weight derived from SSDR [24] and the animation sequence.

We further investigate the relationship between the computational time for solving Eq. (7) and two variables: the number of deformed poses (#Frames) and the number of mesh vertices (#Vertices) in the Hulk model. To analyze the impact of #Frames on running time, we

Table 6

Time required for each stage of EasySkinning. User: user types (Pro: professional, Nov: novice). T1: rigid weight initialization, T2: curve editing, T3: smooth weight solving.

Models	User	T1 (s)	T2 (s)	T3 (s)	Total time (s)
Brute	Pro	0.398	376	15.895	392.293
Hulk	Nov	0.120	523	7.497	530.617
Claire	Pro	0.117	260	9.304	269.421
Amy	Nov	0.350	152	12.957	165.307
Ninji	Nov	0.342	388	13.692	402.034
Tiger	Nov	0.105	345	8.011	353.116
Dog	Nov	0.072	297	2.964	300.036
Horse	Nov	0.210	364	5.179	369.389

vary the step size during sampling of rotation angles, resulting in different lengths of animation sequences. Additionally, to explore how #Vertices influences the solving time, we employ the midpoint subdivision algorithm [31] to generate multiple mesh resolutions. Fig. 11 displays the relationship between the solving time and these two variables, it indicates the solving time is linearly proportional to both #Frames and #Vertices. This suggests that increases in either the number of frames or mesh resolution directly influence the time required for smooth weight solving. Meanwhile, Fig. 12 demonstrates that deformation quality improves with an increase in the length of the animation sequence, but the improvements diminishes as the sequence length becomes longer. This highlights the importance of choosing appropriate animation length to balance between deformation outcomes and solving time.

5.2. User study

Three user studies were conducted to evaluate the proposed system. The first study is to access the entire workflow shown in Fig. 1. The second study is to measure the deformation quality of our automatic skinning workflow. The third study is to evaluate the proposed curve-editing method compared with other interactive techniques. We recruited eight participants for the study, including two professional riggers with one year of rigging experience and six novices. Participants were introduced to various interfaces through videos and instructions, covering the painting interface [9], spline interface [10], and the proposed EasySkinning system.

In the first study, we provide each participant with a character mesh along with its skeleton and ask him/her to skin the character from the scratch. Fig. 13 displays the characters rigged by participants using the proposed EasySkinning system. Table 6 presents the time required for each stage of our method per participant. A notable observation from Table 6 is the efficiency of our method. On average, users with no rigging experience can fully skin a character within ten minutes.

For the second study, we collaborated with two professional riggers to establish five criteria for evaluating the quality of skinning: no adhesion, no penetration, no cleft, no bulge, and volume preservation,

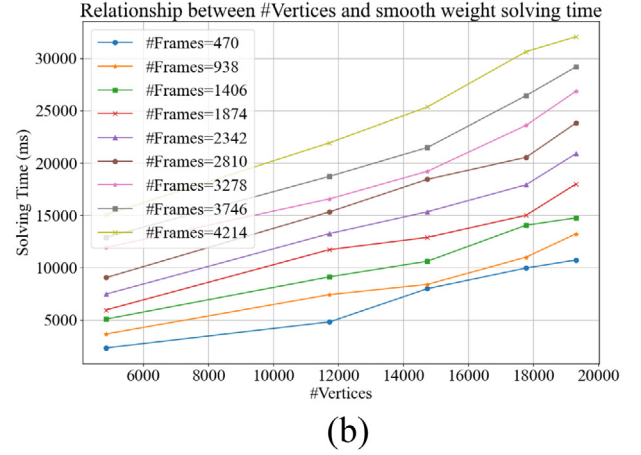
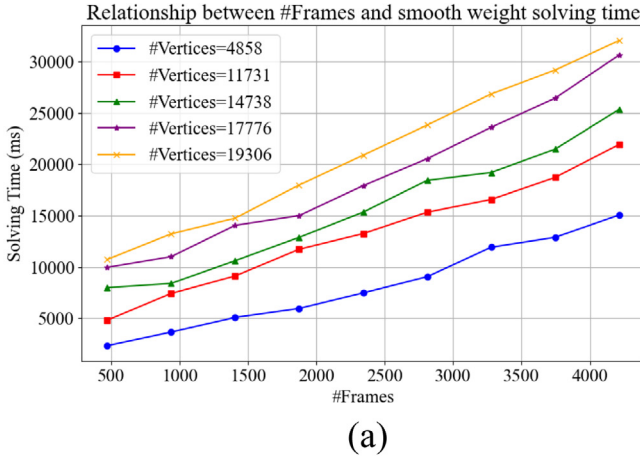


Fig. 11. Smooth weight solving time is linearly proportional to both #Frames and #Vertices. The character is deformed under different poses per frame.



Fig. 12. Relationship between #Frames of animation sequence and Avg Dist (Avg Dist is defined in 5.3.1).

which is illustrated in Table 7. Additionally, we established a standard motion sequence for testing purposes. For a detailed analysis of this study, please refer to Section 5.3.1.

In the third study, each participant was given an auto-skinned character and asked to refine the skinning weights using three different interactive systems: the painting interface [9], the spline interface [10], and the proposed EasySkinning system. This study aimed to compare the efficiency and user-friendliness of the proposed curve-editing method against other interactive techniques. For a detailed analysis of this study, please refer to Section 5.3.2.

5.3. Comparisons

5.3.1. Comparisons with automatic skinning methods

We compare our method with four auto-skinning methods: Closest Distance Skinning (ClosestDist), Bounded Biharmonic Weights (BBW) [2], Geodesic Voxel Binding (GVB) [3], and RigNet [5]. The implementations for ClosestDist and GVB are sourced from Maya, using default parameters. BBW is implemented using libigl,⁴ and for RigNet, we use the officially released code.⁵ In our tests, we exclude the curve-editing step from our method and rely solely on the rigid weights generated by the proposed Voronoi Contraction algorithm.

For a qualitative comparison, we conducted the second user study where users were asked to assess the deformation quality of rigged

characters from Mixamo&Free3D and various auto-skinning methods. During the study, each participant was required to observe the deformation sequences driven by standard motion after skinning (by Mixamo&Free3D, ClosestDist, GVB, RigNet, and our method) and then rate the different methods using a 5-point Likert scale [32]. Table 7 shows that Mixamo&Free3D achieves the best deformation quality across all five criteria, while our method is slightly inferior to Mixamo&Free3D but outperforms the other auto-skinning methods. This is understandable, as the 3D characters of Mixamo&Free3D are hand-rigged, and our automatic workflow aligns mesh vertices with the skeleton better compared to other auto-skinning methods.

To provide a quantitative assessment, we use the skinning weights from Mixamo&Free3D as the ground truth and evaluate the results using the following metrics:

- Average L1 Norm of Skinning Weights (Avg L1): This metric evaluates the deviation between the predicted skinning weight and the ground truth skinning weight (downloaded from the Mixamo and Free3D website, as detailed in Section 5.1). It is calculated as $\frac{1}{n} \sum_i \|w_i - w_i^g\|_1$. A lower L1 value indicates higher accuracy (see Table 2 for symbol definitions).
- Average Deformation Distance (Avg Dist): This metric measures the Euclidean distance between meshes deformed by the predicted smooth weights and the ground truth weights under identical poses, defined as $\frac{1}{n} \|LBS(w_i, v_i) - LBS(w_i^g, v_i)\|$, where LBS denotes the Linear Blend Skinning function (Eq. (5)). A smaller Dist signifies better deformation quality.

During the evaluation of Avg Dist, we randomly rotate each bone 100 times, following a uniform distribution within the range of $[-\pi/6, \pi/6]$. We evaluate these methods on eight characters shown in Fig. 13. Table 8 summarizes the comparative results; the BBW row is left blank as this method requires watertight meshes, which not all our test data satisfy. The quantitative analysis demonstrates that our method outperforms the alternatives in both Avg L1 and Avg Dist metrics. In addition, we show the skinning weights and its LBS deformations by comparing our method with Closest Distance (Fig. 14), BBW (Fig. 15), GVB (Fig. 16), and RigNet (Fig. 17). These figures reveal that our method generates more natural deformations compared to the other automatic techniques.

5.3.2. Comparison with interactive skinning methods

We compare curve-editing interface with paint-based interface [9] and spline interface [10] by conducting the third user study (refer to Section 5.2). In this study, participants were asked to refine skinning weights originally generated by the Geodesic Voxel Binding (GVB) method [3]. We use Autodesk Maya [9] for the painting interface.

⁴ <https://libigl.github.io/>

⁵ <https://github.com/zhan-xu/RigNet>

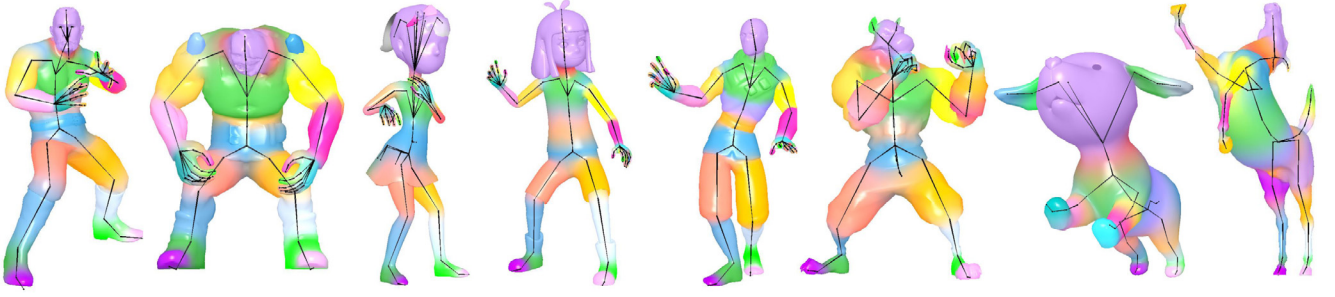


Fig. 13. Characters skinned by the proposed EasySkinning system. The smooth skinning weights are visualized in colorful blend. The deformed vertex positions are calculated using LBS of Eq. (5), which incorporates smooth weights derived from Eq. (7) and skeletal animations from 5.1 as input.

Table 7

Evaluation of skinning quality for Mixamo&Free3D and various auto-skinning methods using a 5-point Likert scale [32], based on the average scores from all participants.

Method	No adhesion ↑	No penetration ↑	No cleft ↑	No bulge ↑	Volume preservation ↑
Mixamo&Free3D	4.92	4	4.5	4.5	4.75
ClosestDist	3.67	3.17	2.75	3.17	2.08
GVB	4.08	3.5	3	3.67	3.25
RigNet	1.25	1	1.17	1.33	1.25
Ours Auto	4.92	3.58	4.25	4.33	4.58

Table 8

Comparisons with other automatic skinning methods. Avg L1 measures the deviation between the predicted skinning weight and the ground truth skinning weight. Avg Dist evaluates the Euclidean distance between meshes deformed by the predicted smooth weights and the ground truth weights under identical poses.

Method	Avg L1 ↓	Avg Dist ↓
ClosestDist	0.754	0.770
BBW	–	–
GVB	0.550	0.554
RigNet	0.580	0.576
Ours Auto	0.342	0.301

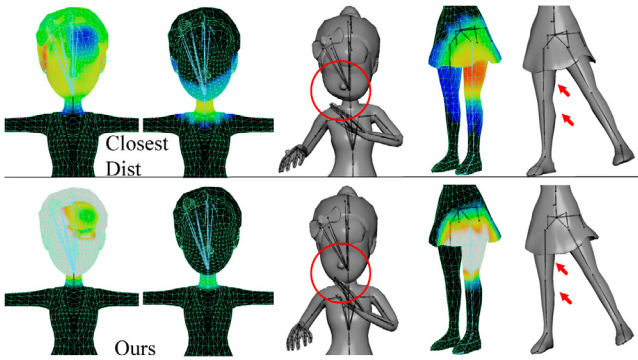


Fig. 14. Skinning weights and LBS deformations on the head and thigh of the Claire model using the ClosestDist (top) and Ours (bottom).

As for the spline interface, we implement functionalities presented in the paper [10]. Unlike paint-based interface and spline interface, our method only requires user to modify rigid weights, therefore, the smooth weights generated by GVB are first converted into rigid weights by binding each vertex to the bone which has the highest influence to it, and then the user can proceed for further editing.

Initial skinning weights computed by GVB are shown in Fig. 18 (1st row). The influence range of the shoulder weights is too large, causing severe inward collapse at the armpits; the neck weights spread to the head, resulting in unreasonable deformations of the mouth vertices along with the neck. Fig. 18 (2nd row), Fig. 18 (3rd row), and Fig. 18

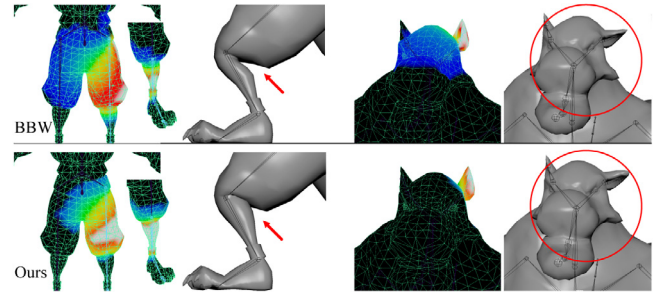


Fig. 15. Skinning weights and LBS deformations on the ankle and ear of the Tiger model using the BBW [2] (top) and Ours (bottom).

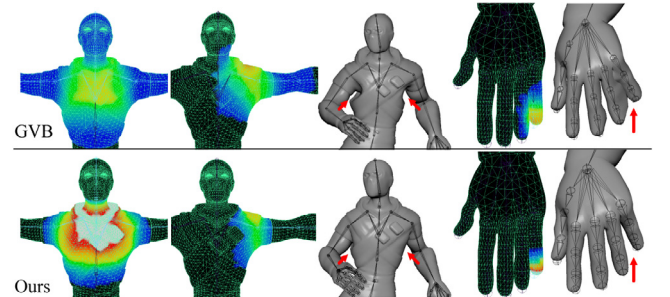


Fig. 16. Skinning weights and LBS deformations on the shoulder and finger of the Ninji model using the GVB (top) [3] and Ours (bottom).

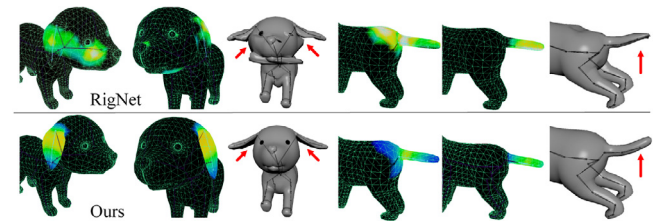


Fig. 17. Skinning weights and LBS deformations on the ears and tail of the Dog model using RigNet [5] (top) and Ours (bottom).

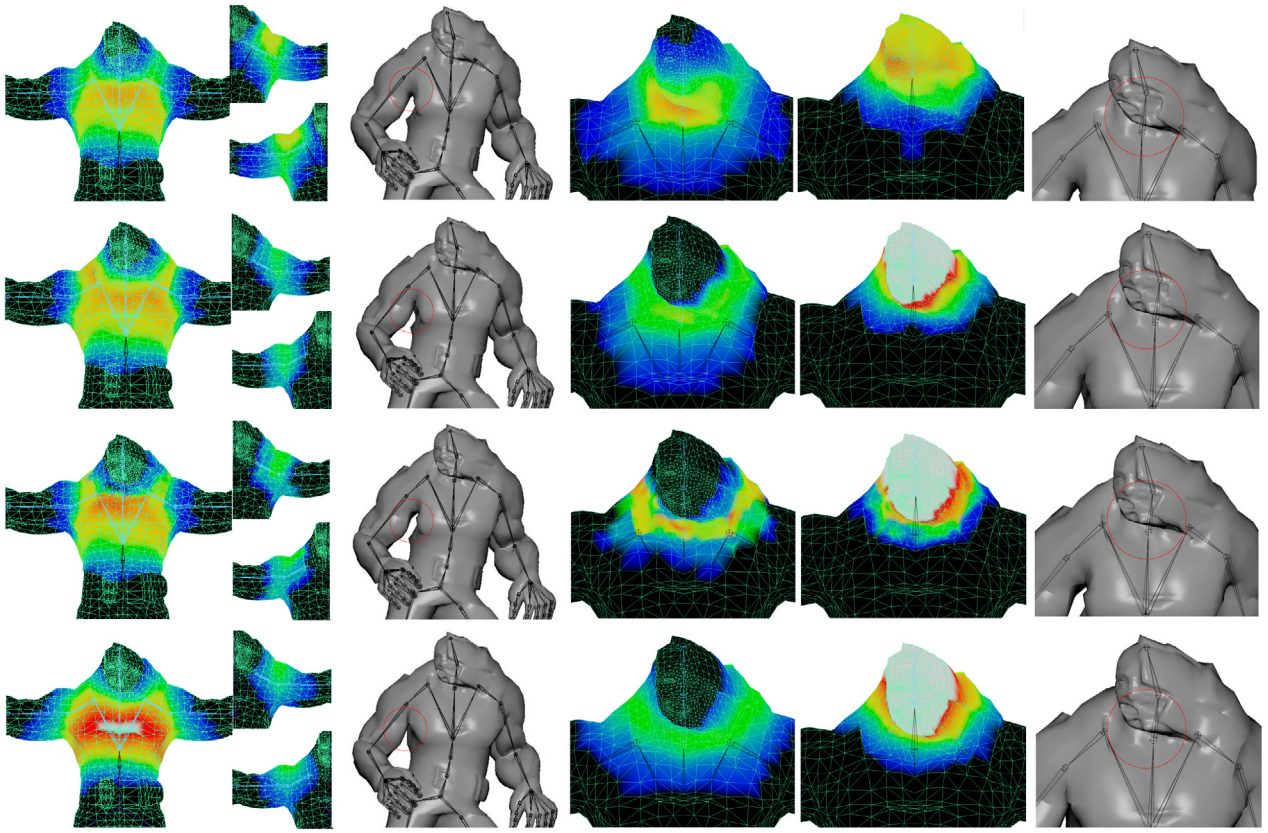


Fig. 18. Hulk (watertight) with automatic skinning using GVB [3] (1st row). Modification of skinning weights through weight painting [9] (2nd row), spline interface [10] (3rd row) and ours (4th row). The corresponding videos are provided in the supplementary.

Table 9

Time comparison with painting interface [9] and spline interface [10]. User: user types (Pro: professional, Nov: novice), T: painting interface, T2: spline interface, T3: our system. Only users assigned to manifold/watertight mesh with single component can use spline interface, Hulk_w: the processed watertight version of Hulk model.

Models	User	T1 (s)	T2 (s)	T3 (s)	T1/T3	T2/T3
Brute	Nov	2953	–	356	8.297	–
Hulk_w	Pro	1810	828	312	5.801	2.654
Claire	Pro	1838	–	272	6.757	–
Amy	Nov	1876	–	247	7.598	–
Ninji	Nov	1905	858	324	5.882	2.648
Tiger	Nov	2819	824	320	8.809	2.575
Dog	Nov	2565	801	299	8.579	2.679
Horse	Nov	2770	819	342	8.099	2.394

(4th row) respectively display the results corrected using the painting interface, spline interface, and EasySkinning. It is evident that our method produces smoother weights and more natural deformations. This improvement is due to our method's approach of deriving weights from the desired deformations, continually optimizing smooth weights to approximate the target deformation, whereas other methods are less consistent in smoothness and quality due to human factors. Fig. 19 shows another experimental result on a Horse model. Similarly, our method achieves more plausible deformations. We record the interaction time of all models in Table 9, which shows that our work achieves 6–8 times speedup over weight painting, and 2–3 times speedup over spline interface.

After completing the user study, we conducted interviews with the participants to gather feedback about their experience with the different interfaces. Many users report that curve-based interface are

more intuitive compared with painting interface. Most novice users found it challenging to adjust the isoline weight value but felt that our method was easier to master than the spline interface. One novice user remarked, “Using the EasySkinning interface feels much more natural; it is like sketching directly on the mesh surface without any other concerns”. However, the professional riggers feel that the curve-based interface, despite intuitive and faster, are not as accurate as the paint-based interface. “The EasySkinning system can provide a good start to work from”, one professional rigger said. “The curve-based interface will accelerate my work a lot, so I only need to deal with some tricky cases”, the other professional rigger said.

6. Conclusion and future work

This paper presents EasySkinning, a target-oriented approach for skinning production characters based on mesh contraction and curve editing. The methodology of EasySkinning, consisting of contraction-based weight initialization, curve-based editing, and target-driven smooth weight solving, addresses the fundamental challenges in character skinning. Our experiments and comparisons with existing methods have demonstrated the efficacy of EasySkinning in terms of precision, generalizability, and ease of use. The key contributions of our research are threefold.

- We enhance accuracy in the initialization stage by using contraction-based weight initialization and intuitive curve editing tools. This approach ensures precise mapping of mesh vertices to corresponding bones, particularly in intricate mesh areas.
- We establish a general framework for the optimization (diffusion) stage, deriving smooth weights to approximate target deformations, which enhances the realism of character deformations.

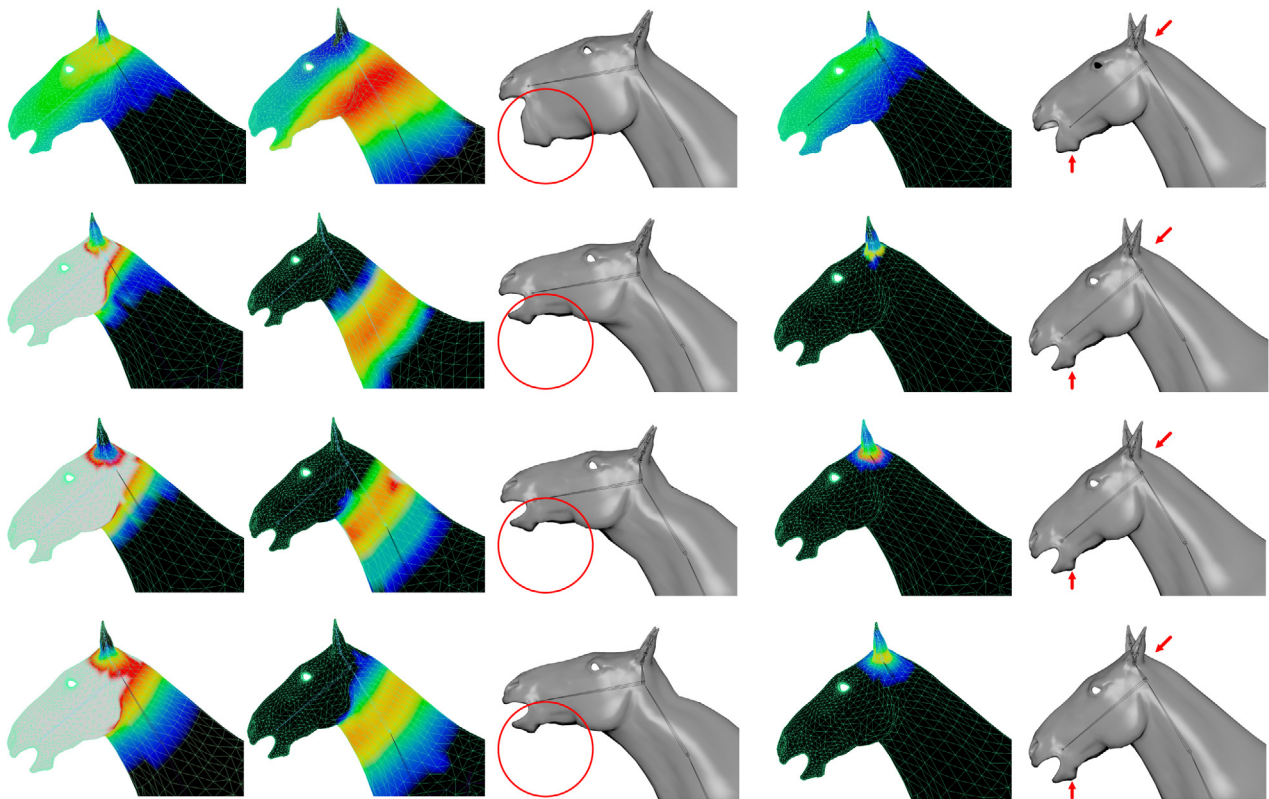


Fig. 19. Horse with automatic skinning using GVB [3] (1st row). Modification of skinning weights through weight painting [9] (2nd row), spline interface [10] (3rd row) and ours (4th row). The corresponding videos are provided in the supplementary.

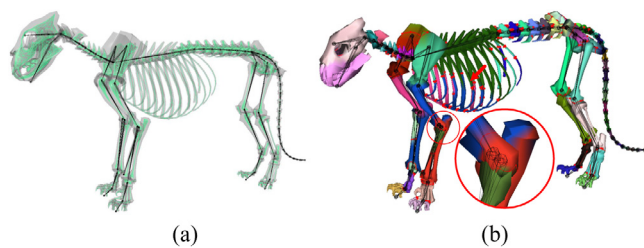


Fig. 20. Failure case.

- We develop an easy-to-use system that is suitable for production meshes with multi-components, self-intersections, and non-manifold primitives, enabling users to achieve greater precision in less time.

Our work has some limitations that could be interesting topics for future work. First, our contraction-based segmentation for rigid weight initialization fails when the skeleton deviates significantly from the mesh, as shown in Fig. 20. Predicting the segmentation/influential region of each bone through deep learning methods and then training a neural network (maybe multimodal large models) to diffuse, namely, achieving smooth weights based on target deformations and visual appearance, could be an important direction for future research. We also raise the question, “Does the scaling law of large language model also apply to tasks requiring great accuracy such as skinning or physical simulation?”, which could also be a valuable research topic.

Second, despite supporting multi-component meshes, our curve-editing tools become less convenient and stable when many small components are clustered together. Developing an efficient and robust curve-editing tool suitable for meshes containing many components

could make this method more applicable to industrial scenarios. A simple approach could involve merging multi-components into a watertight mesh. Users could perform curve-editing on this watertight mesh and then map the edited weights back onto the original mesh

CRediT authorship contribution statement

Jing Ma: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Jituo Li:** Writing – review & editing, Validation. **Dongliang Zhang:** Writing – review & editing, Validation, Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported in part by the National Key Research and Development Program of China (2022YFB3303100).

Appendix A. Projective skinning

See Fig. A.21.

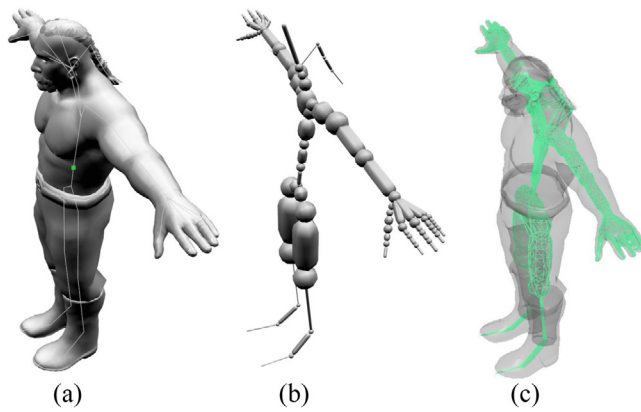


Fig. A.21. Projective Skinning. (a) The non-volumetric skeleton is shown in white line. (b) Volumetric skeleton. (c) The contracted mesh is obtained by intersecting the volumetric skeleton with rays cast from the mesh vertices to base points of the non-volumetric skeleton.

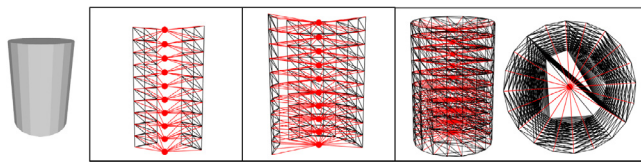


Fig. B.22. The Voronoi centers (right, red points) of a cylinder (left) observed from different cross-sections. Each vertex is connected to its corresponding Voronoi centers by red lines.

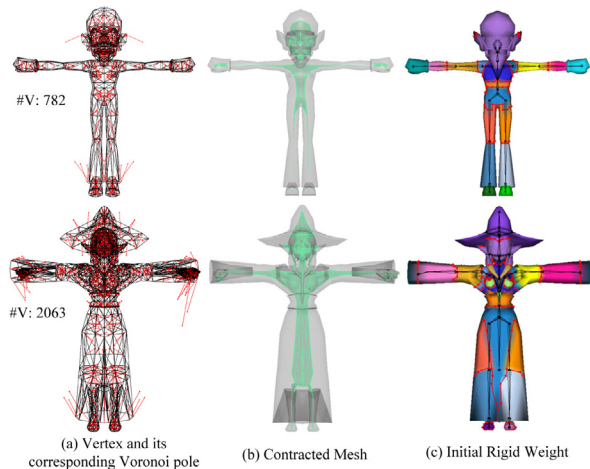


Fig. C.23. Rigid weight initialization on coarse meshes (#V: number of mesh vertices). (a) Connection between a mesh vertex and its corresponding Voronoi pole (red dot) is shown in red line. (b) Contracted mesh obtained by solving Eq. (2). (c) Initial rigid weight.

Appendix B. Voronoi computation

First, we perform Delaunay tetrahedralization on the mesh, dividing its inner region into multiple tetrahedrons. The circumcenter of each tetrahedron is identified as a Voronoi center. Since each mesh vertex belongs to several tetrahedrons, as illustrated in Fig. B.22, we then determine the appropriate Voronoi pole for each vertex. For manifold vertices, we select the nearest Voronoi center as the pole based on the projection distance by projecting the vertex-to-center vector onto the inverse vertex normal. For non-manifold vertices, we choose the vertex itself.

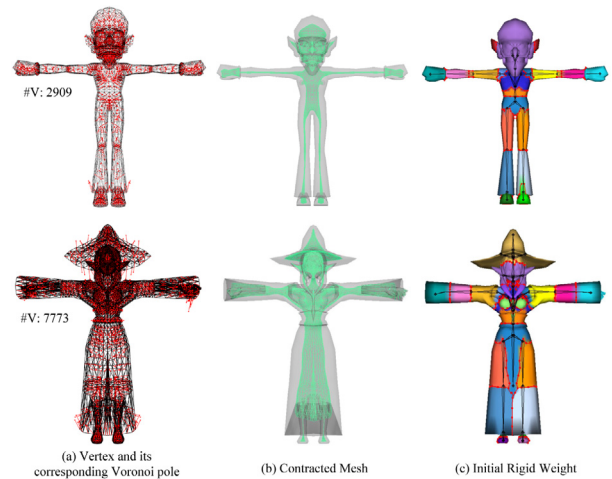


Fig. C.24. Rigid weight initialization on subdivided meshes of Fig. C.23.

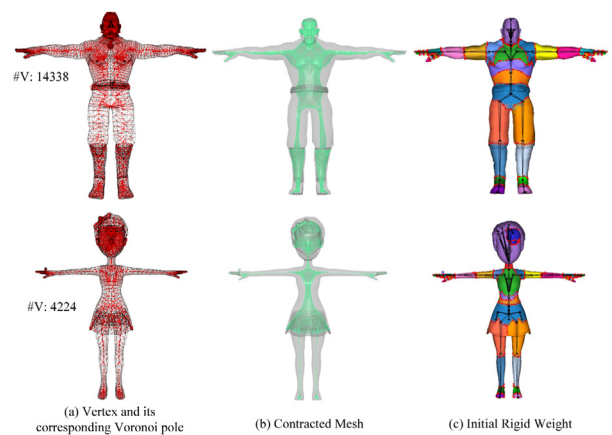


Fig. C.25. Rigid weight initialization on meshes of Fig. 13.

Appendix C. Sensitivity analysis for Section 4.1

We analyze the sensitivity of the proposed Voronoi-guided rigid weight initialization method with respect to mesh resolution. Our method tends to perform less effectively on coarse meshes, as shown in Fig. C.23. However, it yields better results on fine-grained meshes, as illustrated in Figs. C.24 and C.25. To enable users to apply this method on a coarse mesh, we recommend first subdividing the mesh to increase the mesh resolution. Once the rigid weights of the subdivided mesh are obtained, they can be transferred back to the original coarse mesh.

Appendix D. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cag.2024.104049>.

References

- [1] Baran I, Popović J. Automatic rigging and animation of 3d characters. *ACM Trans Graphics (TOG)* 2007;26(3):72–es.
- [2] Jacobson A, Baran I, Popovic J, Sorkine O. Bounded biharmonic weights for real-time deformation. *ACM Trans Graph* 2011;30(4):78.
- [3] Dionne O, de Lasa M. Geodesic voxel binding for production character meshes. In: *Proceedings of the 12th ACM SIGGRAPH/eurographics symposium on computer animation*. 2013, p. 173–80.

- [4] Liu L, Zheng Y, Tang D, Yuan Y, Fan C, Zhou K. Neuroskinning: Automatic skin binding for production characters with deep graph networks. *ACM Trans Graph* 2019;38(4):1–12.
- [5] Xu Z, Zhou Y, Kalogerakis E, Landreth C, Singh K. Rignet: Neural rigging for articulated characters. 2020, arXiv preprint arXiv:2005.00559.
- [6] Li P, Aberman K, Hanocka R, Liu L, Sorkine-Hornung O, Chen B. Learning skeletal articulations with neural blend shapes. *ACM Trans Graph* 2021;40(4):1–15.
- [7] Pan X, Huang J, Mai J, Wang H, Li H, Su T, et al. HeterSkinNet: A heterogeneous network for skin weights prediction. In: *Proceedings of the ACM on computer graphics and interactive techniques*, vol. 4, (no. 1). Association for Computing Machinery; 2021.
- [8] Mosella-Montoro A, Ruiz-Hidalgo J. Skinningnet: Two-stream graph convolutional neural network for skinning prediction of synthetic characters. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, p. 18593–602.
- [9] Autodesk Inc. Paint skin weights. 2022, URL <https://help.autodesk.com/view/MAYAUL/2022/ENU/?guid=GUID-7D773C38-F9CF-4141-8ADD-4E0454838BB7>. [Accessed: 2024-06-12].
- [10] Bang S, Lee S-H. Spline interface for intuitive skinning weight editing. *ACM Trans Graph* 2018;37(5):1–14.
- [11] Igarashi T, Matsuoka S, Tanaka H. Teddy: A sketching interface for 3D freeform design. In: *ACM SIGGRAPH 2006 courses*. 2006, p. 11–es.
- [12] Nealen A, Igarashi T, Sorkine O, Alexa M. Fibermesh: Designing freeform surfaces with 3d curves. In: *ACM SIGGRAPH 2007 papers*. 2007, p. 41–es.
- [13] Liu YJ, Ma CX, Zhang DL. EasyToy: Plush toy design using editable sketching curves. *IEEE Comput Graph Appl* 2009;31(2):49–57.
- [14] Thiery JM, Eisemann E. ARAPLBS: Robust and efficient elasticity-based optimization of weights and skeleton joints for linear blend skinning with parametrized bones. In: *Computer graphics forum*, vol. 37, (no. 1):Wiley Online Library; 2018, p. 32–44.
- [15] Dionne O, De Lasa M. Geodesic binding for degenerate character geometry using sparse voxelization. *IEEE Trans Vis Comput Graphics* 2014;20(10):1367–78.
- [16] Au OK-C, Tai C-L, Chu H-K, Cohen-Or D, Lee T-Y. Skeleton extraction by mesh contraction. *ACM Trans Graphics (TOG)* 2008;27(3):1–10.
- [17] Cao J, Tagliasacchi A, Olson M, Zhang H, Su Z. Point cloud skeletons via Laplacian based contraction. In: *2010 shape modeling international conference*. IEEE; 2010, p. 187–97.
- [18] Sobiecki A, Yasan HC, Jalba AC, Telea AC. Qualitative comparison of contraction-based curve skeletonization methods. In: *Mathematical morphology and its applications to signal and image processing: 11th international symposium, ISMM 2013, Uppsala, Sweden, May 27-29, 2013. Proceedings 11*. Springer; 2013, p. 425–39.
- [19] Komaritzan M, Botsch M. Projective skinning. *Proc ACM Comput Graphics Interact Tech* 2018;1(1):1–19.
- [20] Desbrun M, Meyer M, Schroder P, Barr AH. Implicit fairing of irregular meshes using diffusion and curvature flow. In: *Seminal graphics papers: pushing the boundaries*, volume 2. 2023, p. 149–56.
- [21] Tagliasacchi A, Alhashim I, Olson M, Zhang H. Mean curvature skeletons. In: *Computer graphics forum*, vol. 31, (no. 5):Wiley Online Library; 2012, p. 1735–44.
- [22] De Goeis F, Sheffler W, Fleischer K. Character articulation through profile curves. *ACM Trans Graph* 2022;41(4):1–14.
- [23] Mancewicz J, Derksen ML, Rijpkema H, Wilson CA. Delta mush: Smoothing deformations while preserving detail. In: *Proceedings of the fourth symposium on digital production*. 2014, p. 7–11.
- [24] Le BH, Deng Z. Smooth skinning decomposition with rigid bones. *ACM Trans Graph* 2012;31(6):1–10.
- [25] Schoenberg IJ. Cardinal spline interpolation. SIAM; 1973.
- [26] De Boor C. A practical guide to splines, vol. 27, springer-verlag New York; 1978.
- [27] Unser M. Splines: A perfect fit for signal and image processing. *IEEE Signal Process Mag* 1999;16(6):22–38.
- [28] Sorkine O, Alexa M. As-rigid-as-possible surface modeling. In: *Symposium on geometry processing*, vol. 4, Citeseer; 2007, p. 109–16.
- [29] Le BH, Lewis J. Direct delta mush skinning and variants. *ACM Transactions on Graphics (TOG)* 2019;38(4):1–13.
- [30] Schaefer S, Yuksel C. Example-based skeleton extraction. In: *Symposium on geometry processing*, vol. 153, 2007, p. 162.
- [31] Li T, McKeag R, Armstrong C. Hexahedral meshing using midpoint subdivision and integer programming. *Comput Methods Appl Mech Engrg* 1995;124(1–2):171–93.
- [32] Joshi A, Kale S, Chandel S, Pal DK. Likert scale: Explored and explained. *Br J Appl Sci Technol* 2015;7(4):396–403.